

Pose Estimation of Robot End-Effector using a CNN-Based Cascade Estimator

Kevin D. Ortega^{*}, Jorge I. Sepúlveda^{*}, Byron Hernandez[†], German A. Holguín^{*‡} and Henry, Medeiros[†]

^{*}Dept. Electrical Engineering

Universidad Tecnológica de Pereria, Pereira, Colombia

Email: { kevin.ortega, jorge.sepulveda, gahol, } @utp.edu.co

[†]Dept. Ag. and Bio. Engineering

University of Florida, Gainesville, Florida, USA

Email: { bhernandezosorio, hmedeiros } @ufl.edu

[‡]Dept. Electrical and Computer Engineering

Marquette University, Milwaukee, USA.

Abstract—Robotics has become an essential component of virtually every industry sector, including modern assembly, manufacturing, agricultural processes, and even retail operations. Ensuring the safety of human collaborators working alongside robots is of utmost importance, and accurate estimation of the robot's end-effector pose is critical for achieving this goal. In this paper, we present a method to estimate the end-effector pose of an industrial serial manipulator without relying on the robot's encoders. Our method uses depth cameras in the robot workspace, feeding a convolutional neural network and a cascade estimator to determine the 3D coordinates of every joint sequentially. We studied three variations of the method, with the first estimating all joints independently, the second estimating every joint based on the estimation of the previous joint, and the third using all the previous joints. Our experimental setup employed a UR5 6DOF robot arm in the ROS-Gazebo simulation ecosystem. The results show that the third variation exhibits the best performance, demonstrating the effectiveness of this methodology in predicting the pose of a manipulator using only computer vision. This approach is an enabling technology for many Industry 4.0 applications, improving both human safety and manufacturing efficiency.

Index Terms—Computer vision, Pose Estimation, Robotic Manipulation, Neural Networks, Industry 4.0.

I. INTRODUCTION

In the Industry 4.0 era, where robots and automation are increasingly utilized across various productive sectors, serial manipulators play a significant role in creating efficient human-robot collaborations [1]. Accurate estimation of the pose of the robot end-effector is one of the key tasks in this collaboration. Pose estimation is essential for safely carrying out a variety of robotic tasks, ranging from object manipulation to navigation planning and control, [2]. Unlike traditional industrial robots, which operate in a controlled environment and perform repetitive tasks, robots that work alongside humans need to be able to respond to unstructured and complex scenarios [3], [4].

Robot encoders are the preferred method for determining end-effector pose, but in human-robot collaboration, safety is paramount. Therefore, robots require the ability to handle errors and recover from failures in real-time, be resilient

to uncertainties and failures, and operate safely in dynamic and unpredictable environments [1], [4]. If encoder readings become unavailable or unreliable, the robotic system must have redundancy sources of data to estimate the end-effector pose. This requires advanced sensor fusion algorithms that can integrate multiple sources of sensory information, such as vision, force, and motion, to provide accurate and reliable estimates of the robot's pose.

One alternative method for estimating robot pose are 6DOF laser marker tracker systems. This type of system uses a high-precision laser scanner to track the position and orientation of a reflective marker attached to the robot's end-effector [5]. While it provides accurate and reliable pose estimates, it is a highly complex and invasive system with elevated costs, making it suitable only for calibration and specialized applications. Therefore, for most robot applications, relying on encoders or vision-based methods is a more practical and cost-effective solution.

Our work focuses on developing a computer vision solution for robot end-effector pose estimation to improve the safety of human-robot collaboration, provide redundancy to encoder readings, and enable applications such as collision avoidance and smart path planning. Convolutional neural networks (CNNs) have emerged as highly effective techniques for image regression, enabling practical applications in quality control, object recognition, and process automation in the robotics industry [6], [7]. To improve accuracy, we propose an approach using a cascade of CNNs that simplifies the neural network training process and enhances the accuracy of the robotic arm pose estimation by learning the joint position coordinates of the arm. This solution is expected to enable collaborative robots (cobots) to learn from human interaction and adapt to various work scenarios, enhancing production efficiency and accuracy in identifying the robotic arm pose. Figure 1 presents a simplified overview of our proposed approach, when using 3 RGBD cameras and ResNet50 as the backbone of the pose regressor.

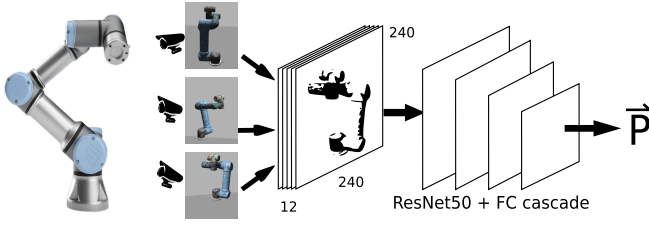


Fig. 1. Proposed System for End-Effector Pose Estimation.

II. STATE OF THE ART REVIEW

The use of cobots in industrial production has been on the rise in recent years due to their numerous benefits, including increased productivity, flexibility, and workplace safety. However, in order to fully realize their potential, it is important to improve human interaction with cobots.

Multi-camera systems are commonly used in computer vision applications as they can significantly improve data accuracy. In a recent study by Zhao et al. [8], a methodology for pose estimation using a three-camera system is proposed, where the main camera has overlapping views with the left and right cameras. This approach uses the optical flow method and keyframe policy to track minutiae and manages 3D points using a Kalman filter. Although this method has good pose estimation accuracy, it may require specific hardware configurations. In contrast, our proposed methodology uses depth cameras in the robot workspace, providing a more economical and flexible solution. Additionally, Parisotto et al. [9] propose an approach that simultaneously addresses object identification and pose estimation. They use a multi-view convolutional neural network (MVCNN) to obtain a view-invariant representation of objects and an object-agnostic deep network to estimate the entropy of viewpoints. The MVCNN then predicts the position and category label of the target object using the obtained viewpoints. In [10], a deep learning-based methodology is proposed for localizing and estimating the joint positions of a robot using a 2D color image. Their approach employs a cascaded convolutional neural network that learns the visual features of the robot's body for localization. The system achieves promising results in terms of localization accuracy and joint estimation in 3D. However, the authors acknowledge that the accuracy is not yet sufficient for visual servoing applications.

On the other hand, Zaatar et al. [1] present a collaborative robot programming framework that incorporates human interaction and deep learning of the dynamics of a production process. This approach is relevant in improving collaboration between humans and robots but does not directly address the accurate estimation of the robot pose.

In another study [11], Romeo et al. in 2021 use a depth-sensing camera (RGBD) and their body tracking library to control cobots in real-time in industrial environments, thereby reducing worker risk. This approach enables the cobot to track the human operator's movements and respond accordingly, allowing for safe and efficient collaboration.

In [12], a sophisticated kinematic error model is proposed that uses the product of exponentials (POE) formula to account for both constant complex geometric errors and joint-dependent errors. To collect position data, a laser tracker and a spherical mounting reflector (SMR) are used, which are installed on the end effector. This innovative approach offers a solution for improving the precision and accuracy of robotic arms. However, the technology used for data acquisition can be costly. In [2], an advanced method for estimating the six degrees of freedom (6-DOF) pose of objects is introduced. This technique leverages a stereo vision system and requires no calibration. Using four registered corners of the object and two cameras, a deep neural network (DNN) is trained to predict the 6-DOF pose based on the detected corner points. This method shows promising results for pick-and-place applications in robotic arms.

Copot et al. [13] recently introduced a cutting-edge visual servo-control algorithm based on position. This algorithm employs a deep neural network to control the UR10 cobot's movements in a vision-based repositioning task. The authors use a dataset generated by simulating the UR10 robot in various positions and capturing images of the object's relative pose in a simulated environment. This approach demonstrates the potential of using advanced machine learning techniques in robotics to improve task performance and efficiency.

III. METHODOLOGY

In our proposed approach, images from all the cameras are combined into a single input tensor. The input tensor is then applied to a ResNet50 [14] architecture used as a backbone to create a feature vector of 1000 elements.

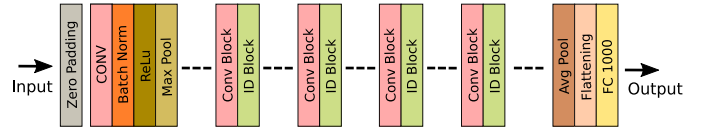


Fig. 2. Architecture of ResNet50.

ResNet50 is a deep convolutional neural network introduced in 2015 within the ImageNet Large Scale Visual Recognition Challenge [14]. An overview of its architecture is shown in figure 2. ResNet50 has become a widely used model architecture, serving as a basis for developing more complex models. The architecture consists of 50 layers and uses skip connections, which allow for the gradient to flow more easily during training and help to alleviate the vanishing gradient problem. ResNet50 also uses global average pooling at the end of the network to reduce the number of parameters and make the network more computationally efficient. Finally, a fully connected layer is used to produce the final output.

Figure 3 shows the overall architecture of our proposed robotic manipulator pose estimation method. The four channels of all the camera images (i.e., RGB-D) are stacked to create the input for the ResNet50 backbone. For example, when using three depth cameras with 240×240 resolution,

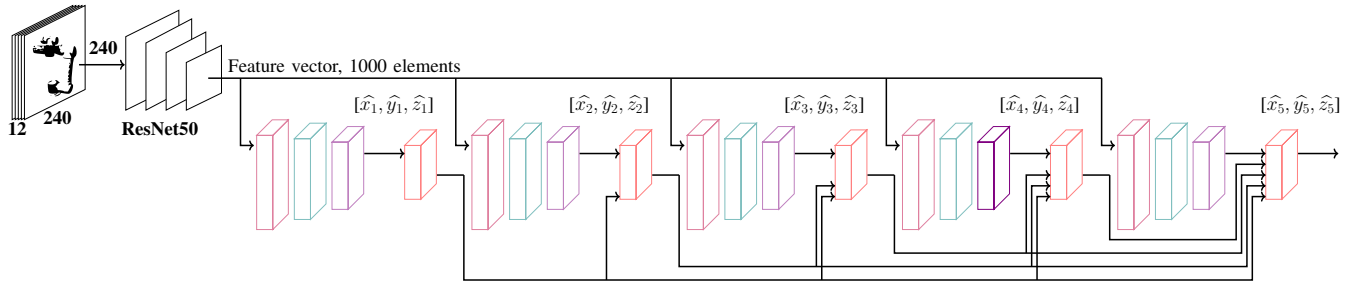


Fig. 3. CNN followed by a fully connected cascade structure.

the input tensor shape will be $12 \times 240 \times 240$. The output of ResNet50 is then given to a cascade of fully connected neural networks, all with the same architecture: 1000 inputs, three hidden layers of 256, 64 and 16 neurons respectively, and 3 outputs as shown in Figure 4.

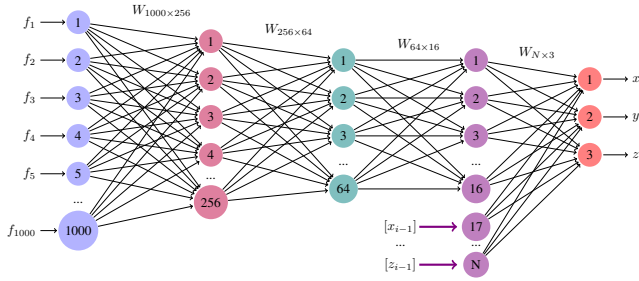


Fig. 4. Fully Connected Block, 1000-256-64-16-3

In section III-A we present the details of our experimental test-bed using the ROS-Gazebo simulation ecosystem. Section III-B present the robot model and kinematics. Section III-C shows the details on the database construction, and section III-D provides the details on our CNN cascade.

A. ROS / Gazebo simulation ecosystem

ROS is an open-source framework that is specifically designed for robotics applications. Gazebo is a 3D virtual reality visualizer that provides a realistic and customizable simulation environment, allowing developers to test and validate their robotic models and algorithms in a safe and controlled setting. ROS, Gazebo, and other tools, such as RViz and MoveIt, create a robotic simulation ecosystem that has become an industry standard and also the de-facto choice for cost-effective robotics research and development.

Figure 5 shows our simulation setup in Gazebo. The UR5 industrial arm was placed in a known position with respect to the origin of coordinates W as shown in the figure. The figure also shows the locations of three depth cameras with the robot within their respective field of view.

Figure 6 displays an example of the RGB images captured by each camera, while Figure 7 illustrates the depth channels of each camera. The depth channel uses the time-of-flight method in the NIR channel to provide estimates of the distances to objects in the scene.

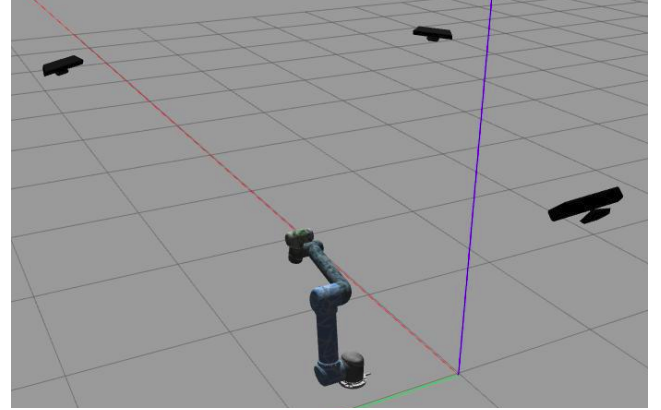


Fig. 5. UR5 arm in the Gazebo simulation environment, including 3 depth cameras in known locations pointing to the arm.

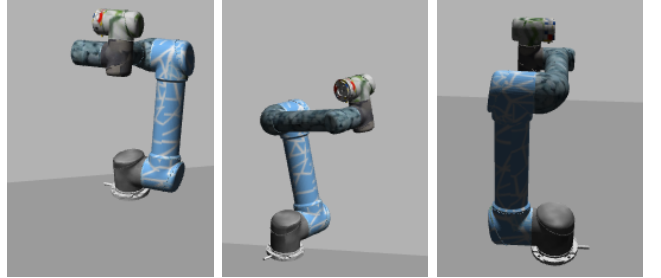


Fig. 6. RGB Images from the cameras.

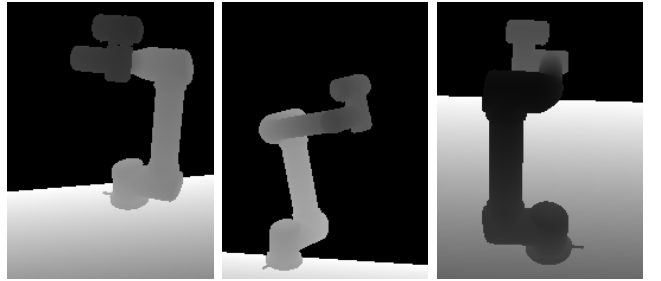


Fig. 7. Depth Images from the cameras.

B. Kinematic description of the manipulator

To fully understand the motion of a serial manipulator, its kinematic parameters must be carefully studied. In this section, we present the kinematic description of a Universal Robots UR5 manipulator, one of the most widely used manipula-

$${}^{i-1}T_i = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) & 0 & a_{i-1} \\ \cos(\alpha_{i-1})\sin(\theta_i) & \cos(\alpha_{i-1})\cos(\theta_i) & -\sin(\alpha_{i-1}) & -d_i\sin(\alpha_{i-1}) \\ \sin(\alpha_{i-1})\sin(\theta_i) & \sin(\alpha_{i-1})\cos(\theta_i) & \cos(\alpha_{i-1}) & d_i\cos(\alpha_{i-1}) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

tors in industrial applications. The Denavit-Hartenberg (DH) representation, which is the most common method used to describe the kinematics of a serial manipulator, is employed to represent the UR5's kinematics. This method uses a set of geometric parameters, such as joint lengths and angles relative to the previous joint, to describe the motion. In Figure 8, the kinematic chain of the UR5 manipulator is detailed. This includes information about the arrangement and orientation of the robot's links and their joints, and how they are connected, namely:

- a_i is the distance between the Z_{i-1} and Z_i axes along X_i ;
- d_i is the distance between the X_{i-1} and X_i axes with respect to Z_{i-1} ;
- θ_i is the angle between the X_{i-1} and X_i axes with respect to Z_{i-1} ;
- α_i is the angle between the Z_{i-1} and Z_i axes with respect to X_i , [15].

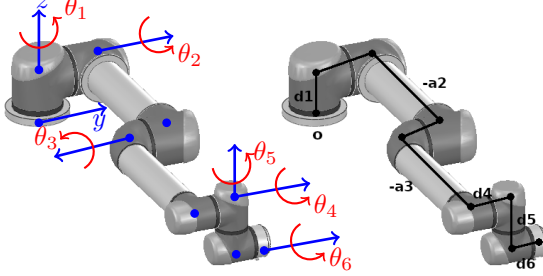


Fig. 8. Kinematic chain for the UR5 robot.

Table I shows the DH parameters of the UR5 manipulator that were used in this study.

TABLE I
DENAVIT-HARTENBERG PARAMETERS FOR THE UR5 MANIPULATOR.

i	α_i	a_i	d_i	θ_i
1	0	0	d_1	θ_1
2	90	0	0	θ_2
3	0	a_2	0	θ_3
4	0	a_3	d_4	θ_4
5	90	0	d_5	θ_5
6	-90	0	d_6	θ_6

To ensure the accurate use of DH parameters for the manipulator, it is a common practice to employ homogeneous transformation matrices (HTMs). These matrices enable the calculation of the transformation from one joint to another as well as the final transformation of the tool [15].

The HTM in equation (1) is used to obtain the transformation between two consecutive joints. The homogeneous transformation matrix T has the structure shown in equation (2), with a rotation part R , and a translation d . F and S are usually zero and one respectively since the transformation should be Euclidean.

$${}^{i-1}T_i = \begin{bmatrix} R_{3 \times 3} & d_{3 \times 1} \\ F_{1 \times 3} & S_{1 \times 1} \end{bmatrix} \quad (2)$$

The end-effector transformation with respect to the origin will be simply the chain product of all individual HTMs, as illustrated in equation (3), [15].

$${}^0T_6 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5 {}^5T_6 \quad (3)$$

For our simulation, we created ROS topics for each 0T_i such that we can access the position of every joint i at any given moment.

C. Database acquisition

As our approach is both supervised and deep, a large annotated dataset is required to train the model effectively. The model takes 3 RGBD images as inputs and predicts the $[x, y, z]$ coordinates of each joint i as outputs. The 3D coordinates of each joint is used later to compute the 6DOF pose of the end-effector $\vec{p} = [x, y, x, \psi, \theta, \phi]$ using the robot kinematics. Table II presents the structure of the collected data.

TABLE II
DATA STRUCTURE FOR ANNOTATED DATASET.

Example	TimeStamp	INPUTS			OUTPUTS (Labels)		
		Cam1	Cam2	Cam3	Position	Orientation	Joints
	t_i	Img1i	Img2i	Img3i	$x \ y \ z$	$\psi \ \theta \ \phi$	$[q1..6]_i$

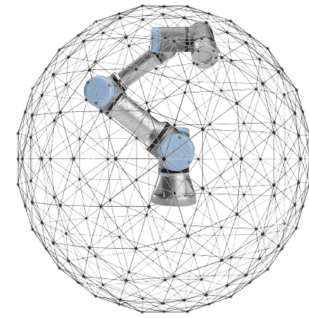


Fig. 9. Reachable spherical workspace.

Figure 9 depicts the reachable workspace of the robot. To construct the dataset, we collected uniformly distributed random samples and filtered out kinematic and Jacobian singularities, which are poses that are either mechanically infeasible to reach (kinematic singularities) or make the Jacobian matrix singular (Jacobian singularities) [15].

In spherical coordinates, each point has three degrees of freedom: the radial distance r , the azimuth angle φ , and the

polar angle θ . If we sample every degree for both φ and θ , we would end up with a dataset of 129,600 data samples. Varying the radial distance r by selecting 10 different values would result in a total of 1,296,000 samples, which can be computationally challenging to handle. To address this issue, we implemented a Monte Carlo sampling strategy that randomly selects 1,200 samples at a time. This allows us to efficiently explore the parameter space and obtain a diverse set of training data, while also reducing the computational burden.

D. CNN model

Our model outputs are the $[x, y, z]$ coordinates of each joint i , as shown in figure 3. We consider three methods for training our network.

- 1) **Baseline:** Fully connected blocks receive only the image features (output of ResNet50).
- 2) **Cascade:** Each fully connected block receives both the image features and the estimates of the immediately previous block, i.e. the estimates of the 3D coordinates of the previous link.
- 3) **Full Cascade:** Fully connected blocks receive the image features and the estimates of all previous blocks.

IV. EXPERIMENTS

We conducted three experiments as described in Section III-D and compared their performance. The implementation was carried out in Python using PyTorch and executed on HiPerGator, the University of Florida supercomputer. The computing node allocated for this experiment consisted of 8 cores and 2 GB of RAM per core, along with an NVIDIA Ampere A100 GPU with 80 GB of VRAM.

Some important parameters during our training were: loss function is MSE (mean square error). Activation in fully connected blocks is the hyperbolic tangent. Learning rate starts with 1×10^{-4} with an exponential decay $\gamma = 0.99$ per epoch. Batch size of 16. Optimizer of choice was RAdam (Rectified Adam), an extension of the Adam optimizer, which combines the benefits of both adaptive learning rate methods like AdaGrad and momentum-based methods like RMSProp.

V. RESULTS

Figure 10 shows the training performance on the network for each method, using the learning rate shown in Figure 11. The baseline method is shown in blue, the cascade method is shown in orange, and the full cascade method is shown in red. From the training perspective all three methods exhibit a similar performance. However, as shown in Figure 12, method three, this is the full cascade, exhibit a better performance than both cascade and the parallel baseline for the validation set. Since the function loss is the MSE, the vertical axes are in $[m^2]$.

A qualitative evaluation can be seen in Figure 13. The ground truth is shown in dotted blue, while the estimated pose is shown in solid red. The estimated pose is perceptually correct, and the average square error is $0.02 m^2$ which is about $0.14 m$ in the Euclidean space.

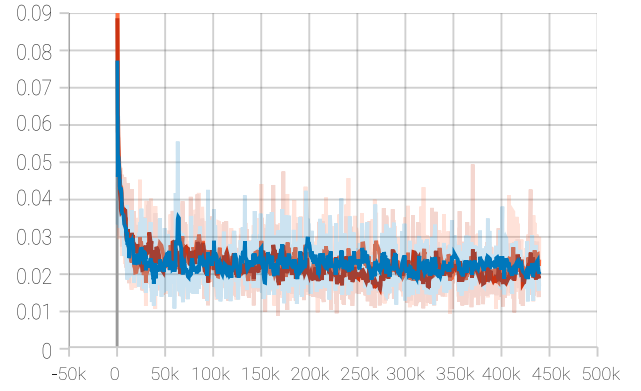


Fig. 10. Training Losses.

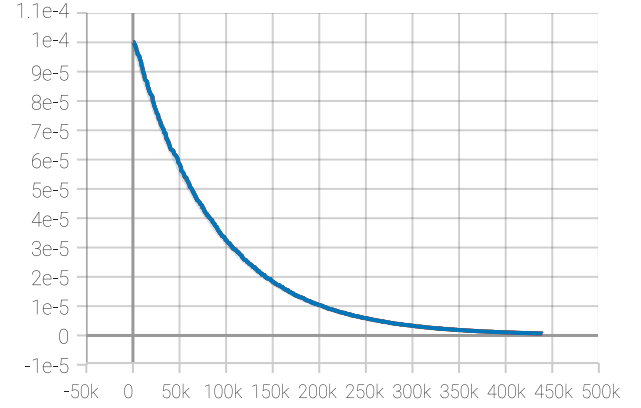


Fig. 11. Exponential Decay Learning Rate.

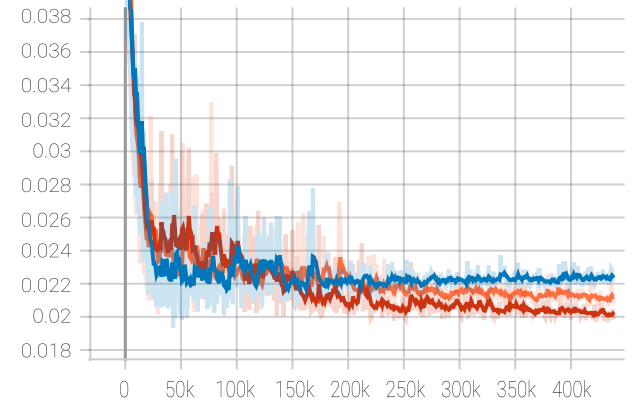


Fig. 12. Validation Losses.

VI. CONCLUSIONS

In this study, we presented and evaluated a method for accurately estimating the pose of a robot's end-effector using only cameras. We compared three variants of the method: a baseline approach employing pose estimation regressors in parallel, a cascade of regressors utilizing the output from the immediately preceding stage, and a full cascade of regressors where each stage incorporates information from all previous stages.

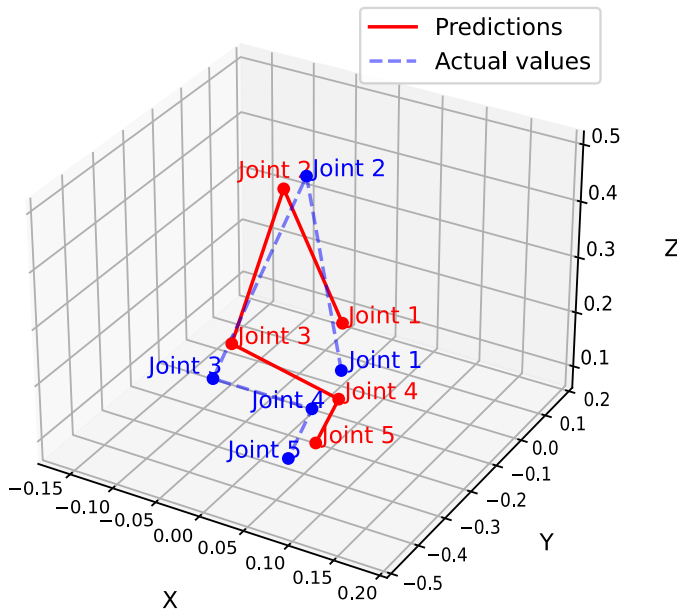


Fig. 13. Qualitative evaluation. Results for one arbitrary pose. Axes in meters.

Among the evaluated variants, the full cascade of regressors demonstrated superior accuracy and effectiveness, resulting in the lowest validation losses. This finding highlights the potential of this approach for practical applications in robotics across a wide range of fields. Unlike laser trackers, which can be invasive, this non-invasive method offers promising prospects for advancing robotics without compromising operational requirements, and at a fraction of the cost.

Furthermore, the qualitative results confirmed the perceptual correctness of the estimated poses. The pose estimations provided by the visual estimator proved to be valuable in cobot applications, enabling tasks such as validating encoder readings, detecting proximity to other objects, and facilitating collision avoidance. These findings underscore the practical utility of the proposed approach in various scenarios, enhancing the capabilities of collaborative robots.

In our future work, we plan to extend this study by conducting experiments on larger datasets to assess the scalability, robustness, and generalizability of the approach. Furthermore, we will conduct an ablation study to investigate the impact of different factors on the model's performance. This study will involve varying the number of cameras used in the system to determine the optimal configuration for achieving accurate pose estimation. Additionally, we will perform fine-tuning of the hyperparameters to optimize the model's performance and enhance its effectiveness in various scenarios.

VII. ACKNOWLEDGEMENTS

Authors Ortega, Sepúlveda, and Holguín want to thank the support from Universidad Tecnológica de Pereira and its Master program in Electrical Engineering in Pereira Colombia. Author Holguín is also affiliated to the department of Electrical and Computer Engineering at Marquette University.

Authors Hernandez and Medeiros are affiliated and supported by University of Florida at Gainesville, Florida, USA.

REFERENCES

- [1] S. El Zaatari, M. Marei, W. Li, and Z. Usman, "Cobot programming for collaborative industrial tasks: An overview," *Robotics and Autonomous Systems*, vol. 116, pp. 162–180, 2019.
- [2] M. Abdelaal, R. M. Farag, M. S. Saad, A. Bahgat, H. M. Emara, and A. El-Dessouki, "Uncalibrated stereo vision with deep learning for 6-dof pose estimation for a robot arm system," *Robotics and Autonomous Systems*, vol. 145, p. 103847, 2021.
- [3] T. Chen, V. Sampath, M. C. May, S. Shan, O. J. Jorg, J. J. Aguilar Martín, F. Stamer, G. Fantoni, G. Tosello, and M. Calaon, "Machine learning in manufacturing towards industry 4.0," *Applied Sciences*, vol. 13, no. 3, 2023.
- [4] E. De Coninck, T. Verbelen, P. Van Molle, P. Simoens, and B. Dhoedt, "Learning robots to grasp by demonstration," *Robotics and Autonomous Systems*, vol. 127, p. 103474, 2020.
- [5] Z. Zhao, X. Wang, J. Chen, and M. Chen, "A generalized kinematic error modeling method for serial industrial robots based on product of exponentials formula," in *2021 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, pp. 1023–1028, IEEE, 2021.
- [6] S. Jin, J. Bang, and J. Lee, "Determination of defects for dynamic objects using instance segmentation," in *2021 21st International Conference on Control, Automation and Systems (ICCAS)*, pp. 739–742, 2021.
- [7] M. Arredondo-Velázquez, J. Diaz-Carmona, A.-I. Barranco-Gutiérrez, and C. Torres-Huitzil, "Review of prominent strategies for mapping cnns onto embedded systems," *IEEE Latin America Transactions*, vol. 18, no. 05, pp. 971–982, 2020.
- [8] C. Zhao, B. Fan, J. Hu, L. Tian, Z. Zhang, S. Li, and Q. Pan, "Pose estimation for multi-camera systems," in *2017 IEEE International Conference on Unmanned Systems (ICUS)*, pp. 533–538, 2017.
- [9] T. Parisotto, S. Mukherjee, and H. Kasaei, "More: Simultaneous multi-view 3d object recognition and pose estimation," *arXiv preprint arXiv:2103.09863*, 2021.
- [10] J. Miseikis, P. Knobelreiter, I. Brijac, S. Yahyanejad, K. Glette, O. J. Elle, and J. Torresen, "Robot localisation and 3d position estimation using a free-moving camera and cascaded convolutional neural networks," in *2018 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pp. 181–187, 2018.
- [11] L. Romeo, R. Marani, M. Malosio, A. G. Perri, and T. D'Orazio, "Performance analysis of body tracking with the microsoft azure kinect," in *2021 29th Mediterranean Conference on Control and Automation (MED)*, pp. 572–577, 2021.
- [12] Z. Zhao, X. Wang, J. Chen, and M. Chen, "A generalized kinematic error modeling method for serial industrial robots based on product of exponentials formula," in *2021 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, pp. 1023–1028, 2021.
- [13] C. Copot, L. Shi, E. Smet, C. Ionescu, and S. Vanlanduit, "Comparison of deep learning models in position based visual servoing," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–4, 2022.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [15] J. J. Craig, *Introduction to robotics*. Pearson Educacion, 2006.